



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊

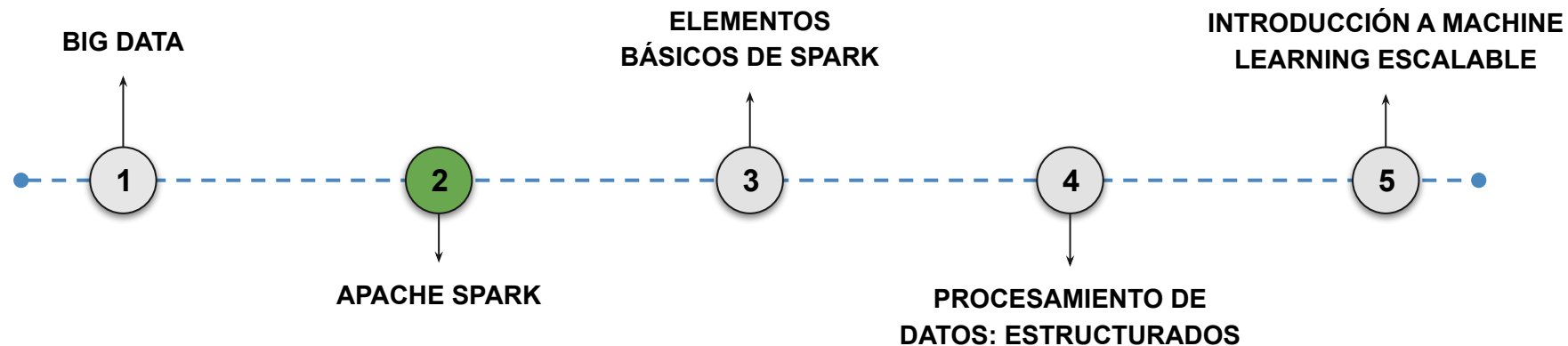


› Apache spark

Aprendizaje Esperado 2: Explicar las características y componentes principales del framework apache spark para el procesamiento de altos volúmenes de datos.

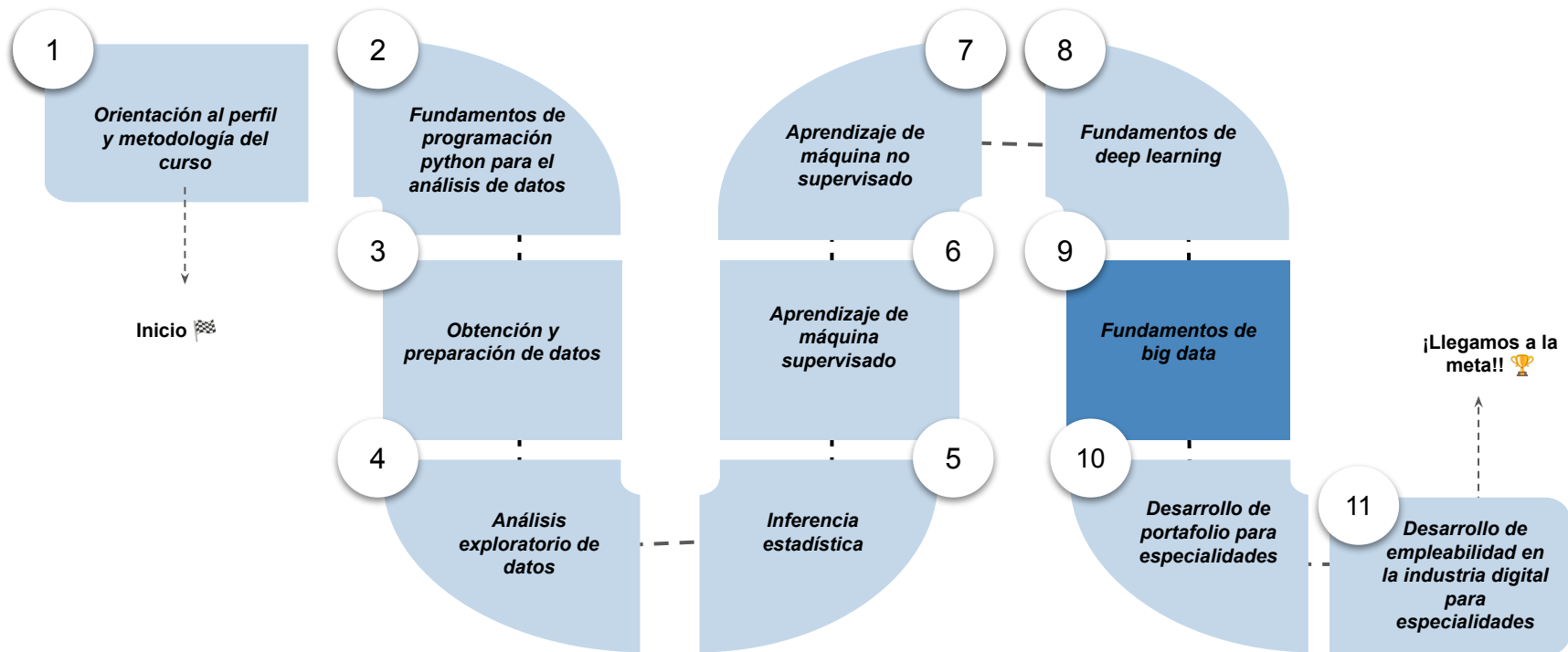
Roadmap de lecciones

¿Cuáles *lecciones* estaremos estudiando en este módulo?



Hoja de ruta

¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



Learning Path

¿Cuáles temas trabajaremos hoy?

2.

Procesamiento distribuido con Apache Spark

Exploraremos los fundamentos de Apache Spark, su arquitectura distribuida, su ecosistema unificado y los problemas que resuelve frente a sistemas tradicionales como Hadoop MapReduce.

¿Qué es Apache Spark y por qué se necesita?

Introducción a Spark y diferencias con Hadoop.

Características clave: procesamiento en memoria, versatilidad, APIs.

Fundamentos técnicos de Apache Spark

RDD, DataFrames, Datasets.

Lazy evaluation y SparkContext.

Learning Path

¿Cuáles temas trabajaremos hoy?

2.

Componentes internos y arquitectónicos de Apache Spark

Analizaremos en profundidad la arquitectura de ejecución de Spark, su funcionamiento en modo clúster y sus distintas implementaciones, como PySpark y servicios en la nube.

Arquitectura de ejecución
distribuida en Spark

Modelo Maestro-Eslavo.

Componentes: Driver,
Executors, Cluster
Manager.

Implementaciones de
Spark

Despliegue en modo
clúster.

Servicios gestionados en
la nube (AWS, GCP,
Azure).



Objetivos de aprendizaje

¿Qué aprenderás?



- Comprender qué es Apache Spark y cómo difiere de Hadoop MapReduce.
- Conocer las ventajas de Spark en términos de rendimiento, escalabilidad y flexibilidad.
- Identificar sus componentes clave: RDD, DataFrames, Datasets, SparkContext.
- Explicar cómo Spark gestiona el procesamiento en memoria y en tiempo real.
- Reconocer cuándo y por qué conviene utilizar Spark en un entorno distribuido.



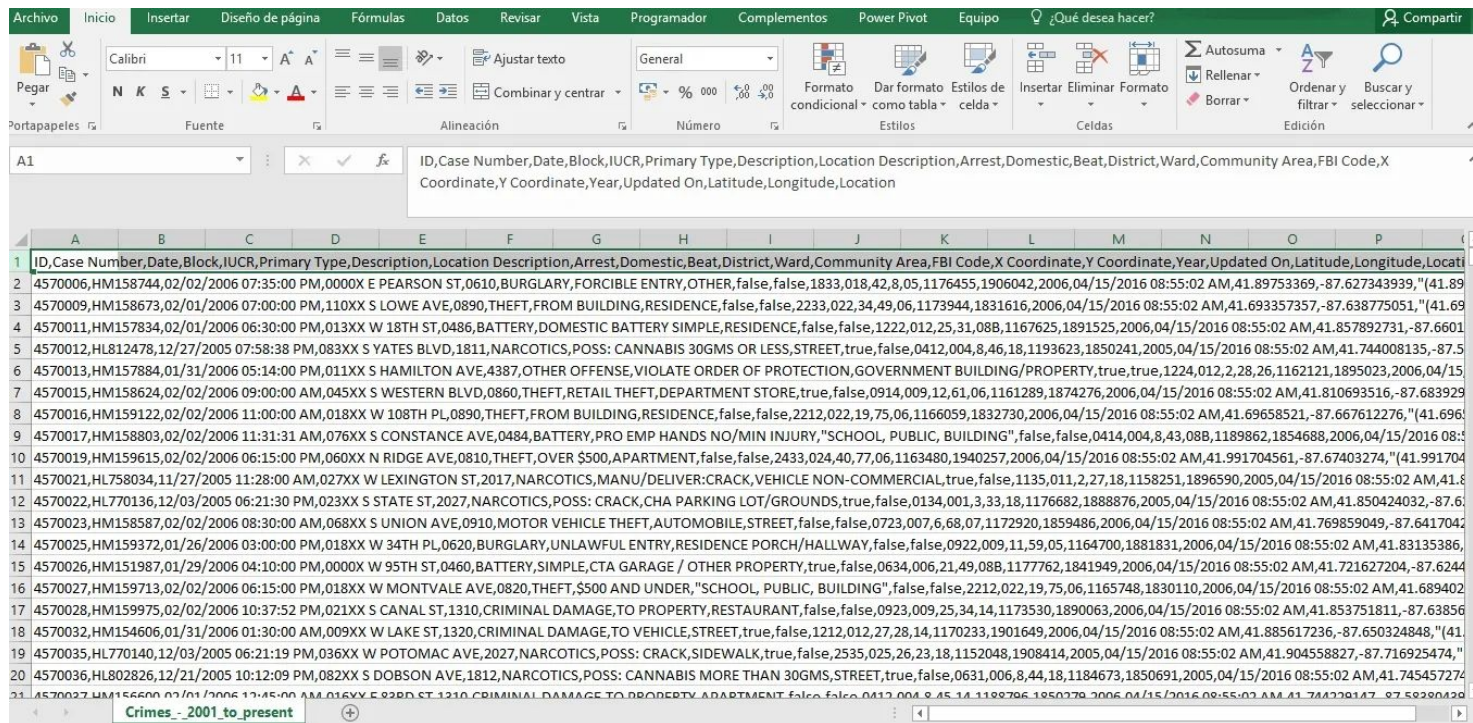
Objetivos de aprendizaje

¿Qué aprenderás?



- Describir la arquitectura interna de Apache Spark y cómo se distribuyen las tareas.
- Explicar el rol del Driver, Executors y el Cluster Manager.
- Analizar cómo Spark divide y ejecuta Jobs en Stages y Tasks.
- Comprender el funcionamiento de Spark en modo clúster.
- Identificar distintas formas de implementar Spark (nube, PySpark, Kubernetes).

Apache spark



Imaginen que tienen un Excel con 500 millones de filas... ¿qué pasa?"

- se cae
- no carga
- es lento

Apache Spark: Procesamiento Distribuido de Datos Masivos

En la actualidad, el análisis de datos masivos y el procesamiento rápido de información se han convertido en necesidades esenciales para empresas e instituciones. La demanda de herramientas que permitan trabajar con grandes volúmenes de datos de manera eficiente ha impulsado el desarrollo de tecnologías como **Apache Spark**.

Apache Spark es un motor de procesamiento distribuido diseñado para manejar grandes conjuntos de datos a alta velocidad. A diferencia de sistemas tradicionales como Hadoop MapReduce, Spark ofrece un rendimiento superior gracias a su capacidad de procesamiento en memoria y su modelo de programación sencillo y flexible.

¿Qué es Apache Spark?

Apache Spark es un motor de procesamiento de datos distribuido y de código abierto, diseñado para ejecutar cargas de trabajo de análisis de datos a gran escala de forma rápida y eficiente.

Spark fue desarrollado originalmente en la Universidad de California, en el laboratorio AMPLab de Berkeley, y en 2014 se convirtió en un proyecto de la **Apache Software Foundation**.

Spark nació como una respuesta a las limitaciones que presentaba el modelo de procesamiento **MapReduce** de Hadoop. Si bien MapReduce fue un avance importante en el tratamiento de datos masivos, su enfoque basado en lectura y escritura a disco después de cada operación generaba una alta latencia y un bajo rendimiento, especialmente en flujos de trabajo iterativos y análisis complejos.

En simple: Spark es una forma de usar muchas máquinas como si fueran una sola

Características principales de Apache Spark

Procesamiento en memoria

La principal diferencia de Spark respecto a sus predecesores es su capacidad para realizar el procesamiento de datos **en memoria (in-memory)**. Esto significa que los datos son almacenados en la memoria RAM de los nodos del clúster mientras se ejecutan las operaciones, evitando costosos accesos al disco y logrando una mejora significativa en los tiempos de ejecución.

Modelo de programación flexible

Además de su alto rendimiento, Spark destaca por su **modelo de programación sencillo y flexible**. Ofrece una interfaz unificada que permite trabajar con diferentes lenguajes de programación, como **Scala, Java, Python y R**, facilitando su adopción por parte de desarrolladores y analistas de datos.

Versatilidad en cargas de trabajo

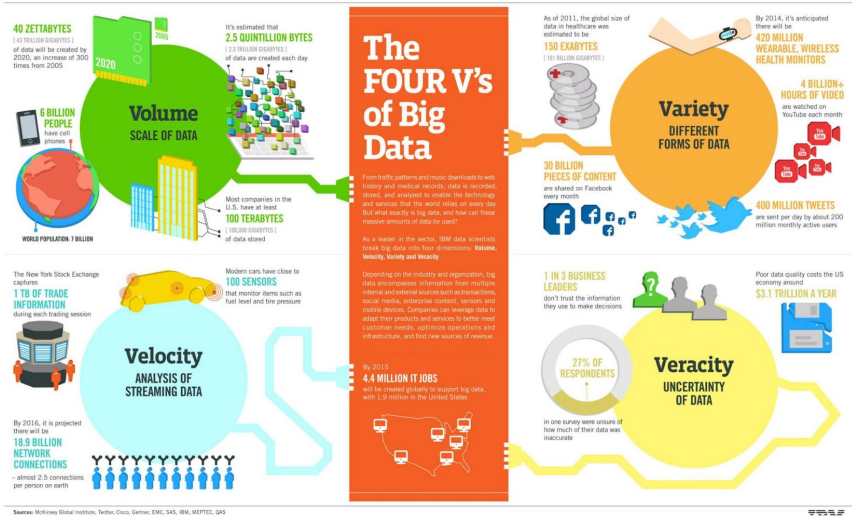
Otro aspecto importante de Spark es su capacidad para manejar diversos tipos de cargas de trabajo. No solo permite el procesamiento por lotes, sino también el análisis de flujos de datos en tiempo real, consultas SQL interactivas, aprendizaje automático y análisis de grafos, todo dentro de un mismo ecosistema.

Necesidad de Apache Spark

El crecimiento exponencial de los datos generados por empresas, redes sociales, sensores y dispositivos IoT ha incrementado la necesidad de soluciones eficientes y escalables para el análisis de datos masivos.

Spark responde a esta necesidad ofreciendo una plataforma robusta, distribuida y de alto rendimiento.

En resumen, Spark es una herramienta esencial en el ecosistema de Big Data, ya que permite a las organizaciones procesar grandes volúmenes de datos de manera rápida y flexible, adaptándose a las demandas de análisis en tiempo real y procesamiento avanzado.



Conceptos básicos de Spark: RDD

Para comprender el funcionamiento de Apache Spark, es necesario conocer algunos conceptos básicos que forman la base de su arquitectura y operación.

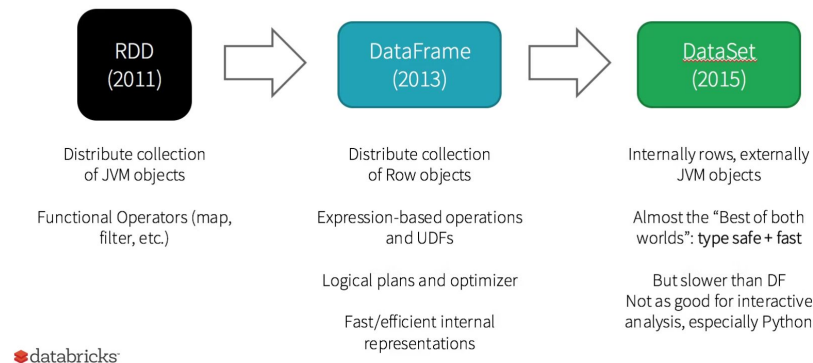
Estos conceptos permiten entender cómo Spark gestiona, procesa y distribuye los datos en un clúster.

Uno de los conceptos clave en Spark es el de **RDD (Resilient Distributed Dataset)**.

Un RDD es una colección inmutable de objetos distribuidos a través de los nodos de un clúster.

Los RDDs permiten realizar operaciones de transformación y acción de forma paralela, garantizando la tolerancia a fallos mediante un mecanismo llamado **lineage**, que permite reconstruir los datos en caso de pérdida.

History of Spark APIs



Piensen en un dataset dividido en pedazos, repartido en varias máquinas

Conceptos básicos de Spark: DataFrames y Datasets

1

DataFrames

Los **DataFrames** son una colección distribuida de datos organizados en columnas, similar a una tabla en una base de datos relacional.

- Estructura tabular con nombres de columnas
- Optimizaciones automáticas mediante Catalyst Optimizer
- Interfaz similar a SQL para consultas

2

Datasets

Los **Datasets** combinan la eficiencia de los RDDs con la simplicidad de los DataFrames, ofreciendo tipado fuerte y validación en tiempo de compilación.

- Verificación de tipos en tiempo de compilación
- API orientada a objetos
- Mejor rendimiento que los RDDs tradicionales

Conceptos básicos de Spark: Evaluación perezosa

Otro concepto importante es el **Lazy Evaluation** o evaluación perezosa. En Spark, las transformaciones aplicadas sobre los RDDs, DataFrames o Datasets no se ejecutan inmediatamente, sino que se almacenan como un plan lógico que se optimiza y ejecuta cuando se solicita una acción, como contar elementos o recolectar resultados.

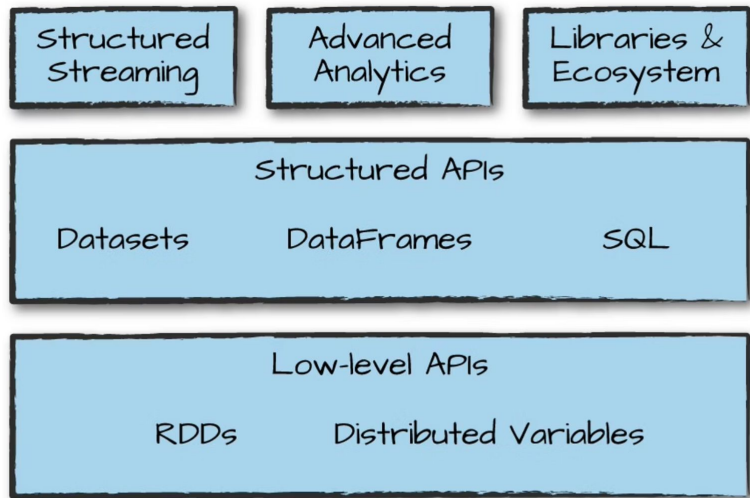


Figure 3-1. Spark's toolset

Este enfoque permite a Spark optimizar el plan de ejecución, minimizando el número de operaciones necesarias y mejorando el rendimiento general del procesamiento de datos.

Modelo de programación de Spark

En Spark hay dos tipos de operaciones:

- **Transformaciones:** Son operaciones donde definimos qué queremos hacer con los datos, pero Spark aún no las ejecuta.
- **Acciones:** Son las operaciones que le piden a Spark un resultado... y ahí recién ejecuta todo

</>

Transformaciones

Las transformaciones, como `map()` o `filter()`, crean un nuevo RDD a partir de uno existente. Estas operaciones son perezosas y no se ejecutan hasta que se solicita una acción.

- `map()`
- `filter()`
- `groupByKey()`
- `join()`

▷

Acciones

Las acciones, como `count()` o `collect()`, devuelven un resultado al programa o escriben datos en un sistema externo. Estas operaciones desencadenan la ejecución del plan de transformaciones.

- `count()`
- `collect()`
- `save()`
- `reduce()`

SparkContext y lenguajes de programación

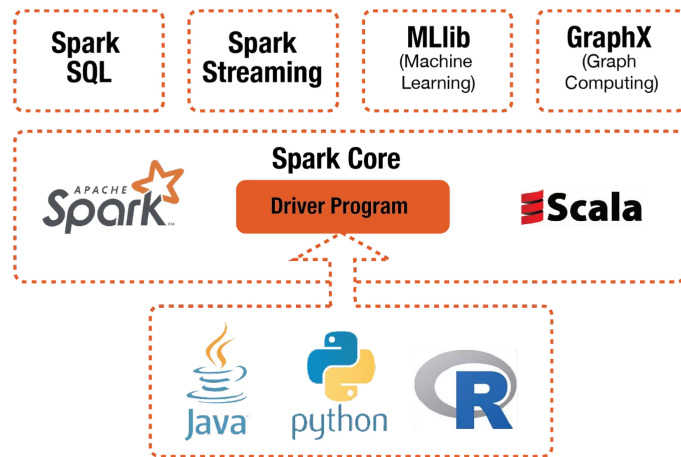
SparkContext es el componente que inicia Spark y permite distribuir datos y tareas en un clúster.

Es el encargado de coordinar el trabajo entre el driver y los workers.

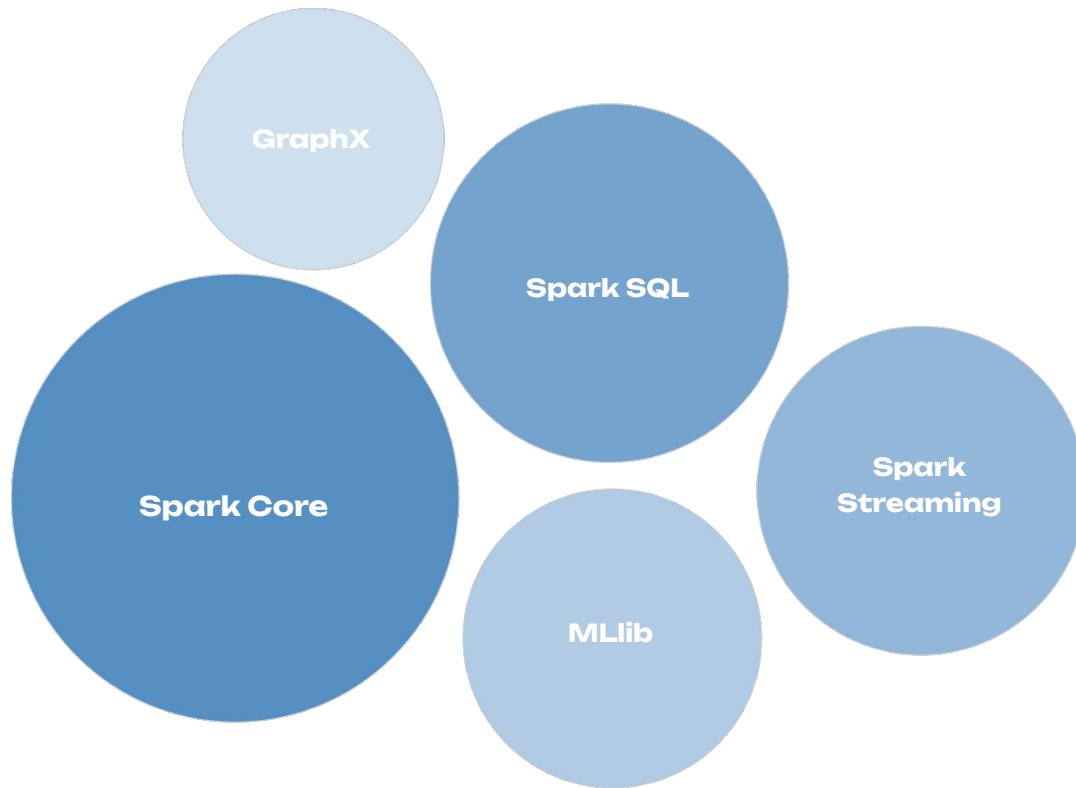
En la práctica, hoy utilizamos SparkSession, que internamente utiliza SparkContext.

Spark permite trabajar con distintos lenguajes:

- Python (PySpark)
- Scala
- Java
- R



Ecosistema unificado de Spark



- Spark Core: Este es el corazón de todo... lo que permite distribuir el trabajo y procesar los datos
- Spark SQL: Para trabajar con datos estructurados, como tablas
- Spark Streaming: Para trabajar con datos en tiempo real
- MLlib: Para machine learning sobre grandes volúmenes de datos
- GraphX: Para trabajar con grafos (menos común en nuestro caso)

Problemas que resuelve Spark:

Latencia alta

Apache Spark fue creado para solucionar varios de los problemas que presentaban los sistemas tradicionales de procesamiento de datos masivos, principalmente aquellos basados en el paradigma MapReduce, como Hadoop.

Uno de los principales problemas que Spark resuelve es la **latencia alta en el procesamiento de datos**. Las soluciones basadas en MapReduce ejecutan cada operación de lectura y escritura sobre disco, lo que genera tiempos de espera elevados. Spark, en cambio, almacena los datos intermedios en memoria (RAM) durante las operaciones, reduciendo drásticamente los tiempos de ejecución.

- Latencia = tiempo de espera

Problemas que resuelve Spark:

Análisis iterativos



Otro problema abordado por Spark es la **dificultad para realizar análisis iterativos**. En tareas como el aprendizaje automático o el análisis de grafos, es común que se requieran múltiples iteraciones sobre el mismo conjunto de datos. Con MapReduce, cada iteración implica leer y escribir datos en disco, mientras que Spark permite mantener los datos en memoria entre iteraciones, mejorando la eficiencia.

Problemas que resuelve Spark:

Procesamiento en tiempo real

En muchos casos, necesitamos procesar datos a medida que llegan, no esperar a procesarlos en lote.

Problema tradicional:

- Procesamiento por lotes (batch)
- Resultados con retraso

Solución con Spark:

- Permite procesar flujos de datos en tiempo real
- Utiliza Spark Streaming
- Facilita casos como:
 - detección de fraude
 - monitoreo en tiempo real
 - personalización de contenido



Problemas que resuelve Spark: Fragmentación de herramientas

Antes de Spark

Las organizaciones debían combinar diferentes tecnologías para cubrir las necesidades de consultas SQL, aprendizaje automático y análisis de grafos.

Con Spark

Spark unifica todas estas capacidades en un solo ecosistema, simplificando el desarrollo y la integración.

Problemas que resuelve Spark:

Complejidad en el modelo de programación

En sistemas tradicionales, procesar grandes volúmenes de datos implicaba escribir código complejo y poco intuitivo.

Problema:

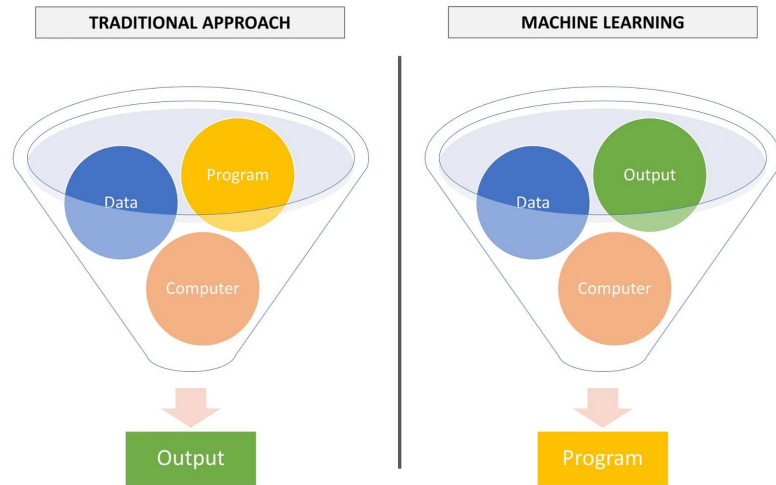
- Código difícil de entender y mantener
- Mayor esfuerzo para desarrollar soluciones

Solución con Spark:

- APIs más simples e intuitivas
- Uso de DataFrames y SQL
- Menor complejidad en el desarrollo

Resultado:

Mayor productividad y facilidad de uso



Problemas que resuelve Spark:

Escalabilidad y recuperación ante fallos

Escalabilidad horizontal eficiente

- Permite procesar más datos agregando más máquinas
- No requiere rediseñar el sistema
- Crecimiento flexible y eficiente

Recuperación ante fallos

- Si una parte del sistema falla, Spark puede continuar
- No es necesario reiniciar todo el proceso
- Reconstruye los datos automáticamente

Comparativa con otros sistemas: Spark vs. Hadoop MapReduce

Aspecto	Hadoop MapReduce	Spark
Procesamiento	Basado en disco	En memoria
Velocidad	Más lento	Más rápido
Facilidad	Complejo	Más simple
Uso	Solo batch	Batch + tiempo real + ML

Comparativa con otros sistemas: Spark vs. Hadoop MapReduce

Modelo de procesamiento

La primera gran diferencia entre Spark y Hadoop MapReduce radica en el modelo de procesamiento. Mientras que **MapReduce** se basa en un procesamiento por lotes que lee y escribe en disco después de cada operación, **Spark utiliza un enfoque de procesamiento en memoria**. Esto le permite realizar las operaciones de manera mucho más rápida, reduciendo drásticamente la latencia y los tiempos de ejecución.

Versatilidad

Otra diferencia clave es la **versatilidad**. Hadoop MapReduce está limitado al procesamiento por lotes, mientras que Spark soporta diferentes tipos de cargas de trabajo en un solo ecosistema: procesamiento por lotes, consultas SQL interactivas, aprendizaje automático, procesamiento en tiempo real y análisis de grafos. Esto hace que Spark sea una solución más completa y flexible.

Facilidad de uso

En cuanto a la facilidad de uso, Spark ofrece una **API más sencilla e intuitiva** que MapReduce. El modelo de programación de MapReduce es complejo y requiere escribir un gran volumen de código para tareas relativamente simples. Por el contrario, Spark proporciona APIs en múltiples lenguajes (Scala, Java, Python y R) y un modelo funcional que facilita el desarrollo.

Comparativa con otros sistemas:

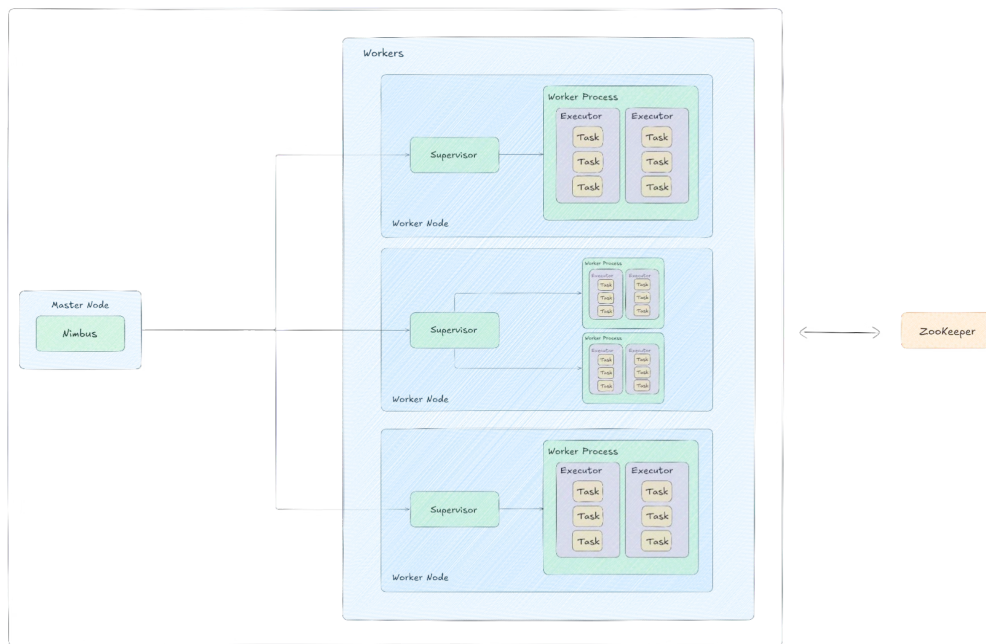
Spark vs. Apache Flink

Comparando Spark con **Apache Flink**, ambos sistemas permiten el procesamiento en tiempo real y por lotes.

Sin embargo, Flink fue diseñado específicamente para flujos de datos continuos y ofrece una arquitectura **nativa para streaming**, mientras que Spark originalmente estaba orientado a procesamiento por lotes y añadió Spark Streaming como un módulo adicional.

Feature	Spark	Flink	Macrometa
Data Processing	Batch/Stream (Micro Batch)	Batch/Stream (Native)	Batch/Stream
Iterations	No	Yes	Yes
SQL	Spark SQL	Table, SQL API	SQL, Javascript
Fault tolerance	Yes (WAL)	Yes (Chandy-Lamport)	Yes (WAL)
Optimization	Manual	Auto	Auto
Windowing	Timed	Time, Count	Time, Count, Sliding, Tumbling
State Backend	HDFS	In-memory, file system, RocksDB	In-memory, multi-model DB
Language	Scala, Java, Python, R, C#, F#	Java, Scala, Python, SQL	SQL, Javascript
Cost	Open source	Open source	Commercial
Backpressure	Manual Configuration	Implicit, through architecture	Implicit
Geo-Distribution	No	No	Yes
Latency	Seconds	Sub-second	Sub-second
Geo-fencing	No	No	Yes
Converged Platform	No	No	Yes

Comparativa con otros sistemas: Spark vs. Apache Storm



Respecto a **Apache Storm**, este sistema también permite el procesamiento en tiempo real, pero está más enfocado en tareas de baja latencia y eventos individuales.

Spark Streaming, en cambio, trabaja por micro-lotes, lo que puede introducir cierta latencia adicional, aunque permite una integración más sencilla con el resto del ecosistema Spark.

Comparativa: Tolerancia a fallos y rendimiento

Tolerancia a fallos

- Spark puede continuar si una parte del sistema falla
- No es necesario reiniciar todo el proceso
- Recupera los datos automáticamente

Rendimiento

- Spark procesa datos más rápido (uso de memoria)
- Especialmente eficiente en procesos iterativos
- Reduce tiempos de ejecución

Comparativa: Tolerancia a fallos y rendimiento

Tolerancia a fallos

Otra diferencia relevante es la **gestión de tolerancia a fallos**. Tanto Spark como Hadoop y Flink cuentan con mecanismos robustos para recuperarse ante fallos de nodos o procesos. Spark utiliza su modelo de RDD y el concepto de lineage para reconstruir los datos, mientras que Flink ofrece un sistema de snapshots y Storm requiere configuración adicional para lograr tolerancia a fallos.

Rendimiento

En términos de **rendimiento**, múltiples estudios han demostrado que Spark supera a Hadoop MapReduce en la mayoría de los escenarios, especialmente en cargas de trabajo iterativas y de aprendizaje automático. En comparación con Flink y Storm, la elección dependerá de los requisitos específicos de latencia y volumen de datos.

Conclusión de la comparativa

En conclusión, Apache Spark se destaca por ofrecer un ecosistema unificado, alto rendimiento y flexibilidad para abordar diferentes tipos de procesamiento de datos, posicionándose como una de las soluciones más completas y adoptadas en el ámbito de Big Data.

Cuándo conviene usar Spark:

Grandes volúmenes de datos

Apache Spark es una herramienta poderosa, pero no siempre es la solución ideal para todos los proyectos. Es importante conocer en qué situaciones y contextos resulta conveniente utilizar Spark para aprovechar al máximo sus capacidades y evitar un uso ineficiente de los recursos.

Spark es recomendable cuando se requiere **procesar grandes volúmenes de datos** que superan la capacidad de los sistemas tradicionales o de un solo servidor. Gracias a su arquitectura distribuida, Spark permite dividir y procesar datos masivos en múltiples nodos de un clúster, asegurando escalabilidad y eficiencia.



Cuándo conviene usar Spark: Análisis complejos e iterativos



Aprendizaje automático

Algoritmos que requieren múltiples pasadas sobre los mismos datos, como clasificación, regresión y clustering.



Análisis de grafos

Procesamiento de relaciones complejas entre entidades, como redes sociales o sistemas de recomendación.



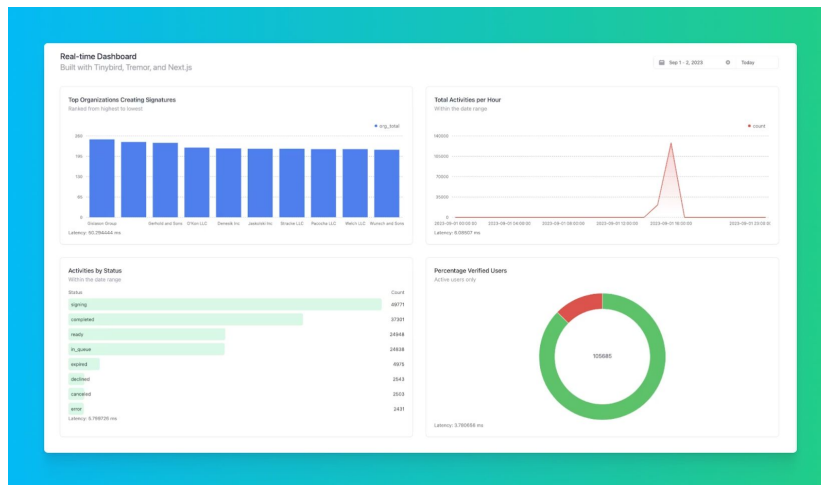
Minería de datos

Descubrimiento de patrones y extracción de conocimiento a partir de grandes conjuntos de datos.

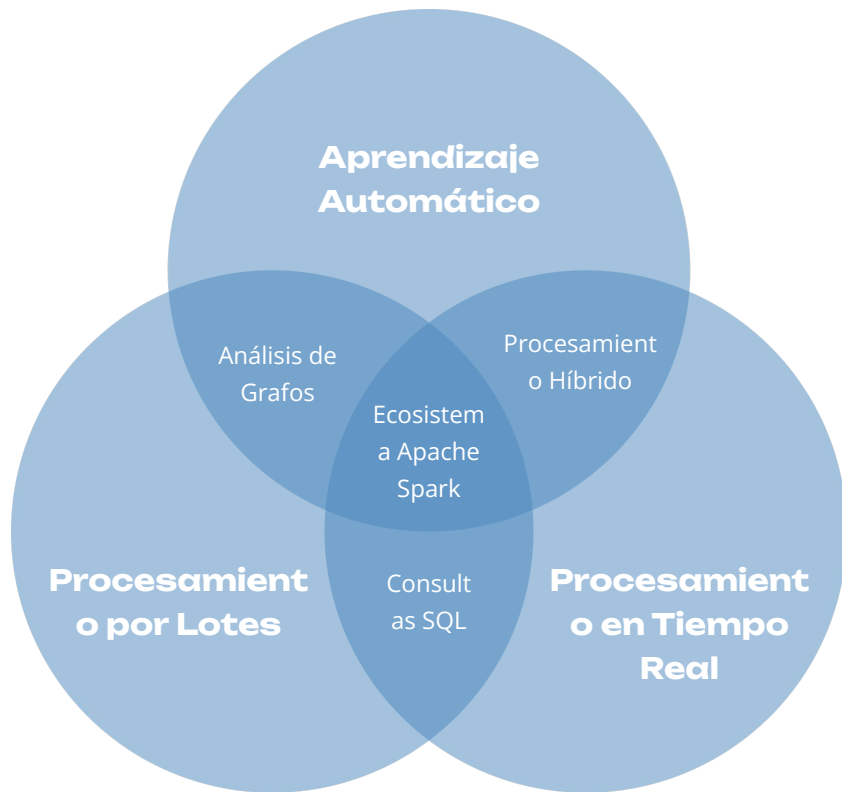
Otra situación ideal para utilizar Spark es cuando las tareas de procesamiento requieren **análisis complejos e iterativos**, como los algoritmos de aprendizaje automático y minería de datos. Spark, al trabajar con datos en memoria, reduce drásticamente los tiempos de ejecución frente a sistemas que dependen de la lectura y escritura en disco.

Cuándo conviene usar Spark: Procesamiento en tiempo real

El uso de Spark también es recomendable en entornos donde se necesita **procesamiento en tiempo real o cuasi tiempo real**. Mediante su módulo **Spark Streaming**, Spark permite capturar y procesar flujos de datos provenientes de sensores, redes sociales, sistemas financieros, entre otros, facilitando la toma de decisiones inmediata.



Cuándo conviene usar Spark: Ecosistema unificado



Cuando un proyecto requiere integrar diferentes tipos de procesamiento —como consultas SQL, aprendizaje automático, procesamiento de datos estructurados y no estructurados, y análisis de grafos— Spark es una opción adecuada, ya que ofrece un ecosistema unificado para cubrir todas estas necesidades.

Cuándo conviene usar Spark:

Experiencia del equipo y entornos distribuidos

Experiencia del equipo

Spark es especialmente útil en proyectos donde el equipo de trabajo ya tiene experiencia en lenguajes como **Scala, Java, Python o R**, ya que su API soporta estos lenguajes y facilita el desarrollo de soluciones personalizadas.

Entornos distribuidos

En entornos donde la infraestructura es distribuida, como clústeres locales, servidores en la nube o sistemas híbridos, Spark es una herramienta adecuada gracias a su capacidad de adaptarse y aprovechar los recursos disponibles de manera eficiente.

Cuándo NO conviene usar Spark

Sin embargo, Spark no es la mejor opción para proyectos con conjuntos de datos pequeños o medianos que pueden ser procesados fácilmente con herramientas tradicionales o bases de datos relacionales. En estos casos, el despliegue y gestión de un clúster Spark puede resultar excesivo y costoso.



Conclusión sobre cuándo usar Spark

En conclusión, Spark conviene ser utilizado en contextos donde se requiere **alto rendimiento, escalabilidad, análisis complejos y procesamiento en tiempo real**. La elección adecuada dependerá siempre de los objetivos del proyecto, el volumen de datos y los recursos disponibles.



Live Coding

¿En qué consistirá la Demo?

En esta demostración se trabajará con un conjunto de datos simple utilizando PySpark, con el fin de analizar cómo Spark procesa la información.

Se crearán estructuras de datos (DataFrames y RDD), sobre las cuales se aplicarán distintas operaciones. A lo largo del ejercicio, se observará que las transformaciones no se ejecutan inmediatamente, sino que Spark construye un plan de ejecución que solo se materializa cuando se invoca una acción.

1. Crear un DataFrame a partir de un conjunto de datos simple.
2. Aplicar transformaciones como filtrado y selección de columnas.
3. Ejecutar acciones como show() y count() para obtener resultados.
4. Analizar en qué momento Spark ejecuta realmente las operaciones.
5. Repetir el ejercicio utilizando RDD para comparar el comportamiento.



Ejercicio

Explorando Apache Spark

Explorando Apache Spark

Contexto: 🙌

Vas a interiorizar los conceptos clave de Apache Spark mediante una exploración guiada de sus componentes y casos de uso.

Consigna: 📝

1. Explica con tus palabras qué es Apache Spark y qué lo diferencia de MapReduce.
2. Describe los conceptos de RDD, DataFrame y Dataset y señala sus diferencias.
3. Enumera los módulos que componen el ecosistema Spark y para qué se usa cada uno.
4. Investiga tres casos reales o simulados donde Spark sea más conveniente que otras herramientas.

Paso a paso: ⚙️

1. Usa el material de clase como guía base.
2. Resume cada bloque de contenido en tus propias palabras.
3. Organiza los resultados en un cuadro comparativo o infografía.
4. Prepara una breve presentación (oral o visual) para compartir en la próxima clase.

Tiempo 🕒: 45 Minutos



Momento:

Time-out!



5 -10 min.



Apache spark

⚡ Instalación rápida de Spark (en 5 minutos)

Ejecutar tu primer código en PySpark sin complicarte.

Paso 1: Instalar PySpark

Abrir terminal y ejecuta: `pip install pyspark`

Paso 2: Abrir Python

Paso 3: Probar Spark

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()
data = [("Juan", 1200), ("Ana", 800)]
df = spark.createDataFrame(data, ["cliente", "monto"])
df.show()
```

Resultado esperado

Deberías ver algo así:

```
+-----+-----+
|cliente|monto|
+-----+-----+
|  Juan| 1200|
|   Ana|  800|
+-----+-----+
```

API de Spark: Flexibilidad y facilidad de uso

Uno de los aspectos que ha contribuido al éxito y adopción masiva de Apache Spark es la flexibilidad y facilidad de uso de su API (Interfaz de Programación de Aplicaciones). Spark ofrece un conjunto de APIs que permiten a los desarrolladores y analistas de datos interactuar con el motor de procesamiento de manera sencilla y eficiente.

La API de Spark es simplemente la forma en que nosotros interactuamos con Spark usando código.

¿Por qué es fácil de usar?

- Permite trabajar con DataFrames (como tablas)
- Tiene una sintaxis similar a SQL y Python
- No necesitas manejar detalles complejos del sistema distribuido

¿Qué significa que es flexible?

- Se puede usar con distintos lenguajes:
 - Python (PySpark)
 - Scala
 - Java
 - R

API de Spark: Estructuras de datos distribuidas

RDD (Resilient Distributed Dataset)

El núcleo de la API de Spark se basa en la manipulación de estructuras de datos distribuidas. La primera y más fundamental de estas estructuras es el **RDD (Resilient Distributed Dataset)**, una colección inmutable y distribuida de objetos que pueden ser procesados en paralelo. Los RDDs permiten aplicar operaciones de transformación (como map, filter) y acción (como count, collect) sobre los datos.

DataFrame

Los **DataFrames** son colecciones distribuidas de datos organizados en columnas, similares a una tabla en una base de datos relacional. Esta estructura ofrece mayor eficiencia y permite realizar consultas SQL sobre los datos.

Dataset

Los **Datasets** combinan la eficiencia de los DataFrames con el tipado fuerte de los RDDs, ofreciendo un balance entre seguridad y flexibilidad.

Distintas formas de trabajar con datos en Spark, con distintos niveles de abstracción

API de Spark: Spark SQL

Spark también proporciona una API para realizar consultas estructuradas mediante **Spark SQL**. Esta interfaz permite ejecutar consultas SQL estándar sobre DataFrames y Datasets, facilitando el trabajo a los usuarios que ya tienen experiencia en lenguaje SQL.

SQL

```
SELECT cliente, monto  
FROM datos  
WHERE monto > 1000
```

Python

```
df.filter(df["monto"] > 1000).select("cliente", "monto")
```

API de Spark: Spark Streaming

Para el procesamiento en tiempo real, Spark ofrece la API de **Spark Streaming**, que permite procesar flujos de datos en micro-lotes. Esta API es compatible con DataFrames y Datasets, y soporta múltiples fuentes de datos en tiempo real, como Kafka, Flume o sockets TCP.

API de Spark: MLlib y GraphX

MLlib

En el ámbito del aprendizaje automático, Spark incluye la biblioteca **MLlib**, que proporciona algoritmos y herramientas para clasificación, regresión, clustering, reducción de dimensionalidad y evaluación de modelos.

La API de MLlib permite construir y entrenar modelos sobre grandes volúmenes de datos de manera eficiente.

GraphX

Además, Spark ofrece la API de **GraphX** para el procesamiento y análisis de grafos.

Esta API permite modelar y manipular grafos distribuidos, facilitando tareas como la búsqueda de rutas, detección de comunidades y análisis de conectividad.

Resumen de la API de Spark

¿Qué aprendimos?

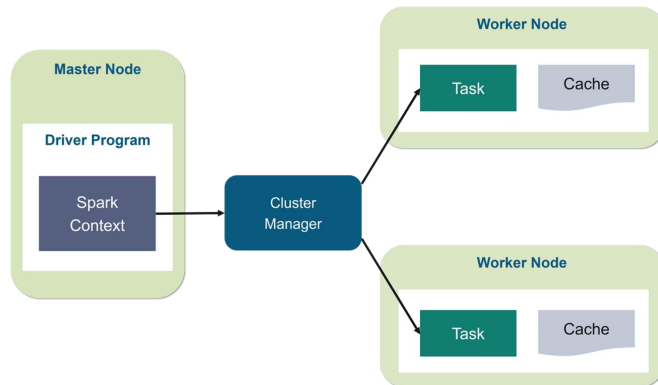
- Spark permite procesar datos de forma distribuida
- Trabajamos principalmente con DataFrames
- Usamos transformaciones y acciones
- Spark ejecuta de forma lazy (bajo demanda)

¿Qué puede hacer Spark?

- Procesamiento de datos (batch)
- Consultas SQL
- Procesamiento en tiempo real
- Machine Learning
- Análisis de grafos

Arquitectura general de Spark

La arquitectura de **Apache Spark** está diseñada para aprovechar el procesamiento distribuido y en memoria, permitiendo manejar grandes volúmenes de datos de manera eficiente y escalable. Comprender su arquitectura es esencial para aprovechar al máximo las capacidades que ofrece este framework.



Modelo Maestro-Esclavo de Spark

Driver (coordinador)

- Ejecuta tu código
- Divide el trabajo en tareas

Workers (ejecutores)

- Ejecutan las tareas
- Procesan los datos

Cluster Manager

- Asigna recursos
- Decide dónde corre cada tarea

El trabajo se distribuye y se ejecuta en paralelo

Modelo Maestro-Eslavo de Spark

Modelo Maestro-Eslavo

La arquitectura de Spark se basa en un modelo **Maestro-Eslavo (Master-Slave)**, en el que existe un nodo principal encargado de la coordinación y varios nodos trabajadores encargados del procesamiento de datos. Este modelo permite distribuir las tareas de procesamiento y garantizar la tolerancia a fallos.

Driver Program

En el centro de la arquitectura se encuentra el **Driver Program**, que actúa como punto de entrada de cualquier aplicación Spark. El Driver es responsable de convertir el código escrito por el usuario en tareas distribuidas y enviarlas a los nodos del clúster para su ejecución. Además, mantiene el estado de la aplicación y coordina la distribución y ejecución de las tareas.

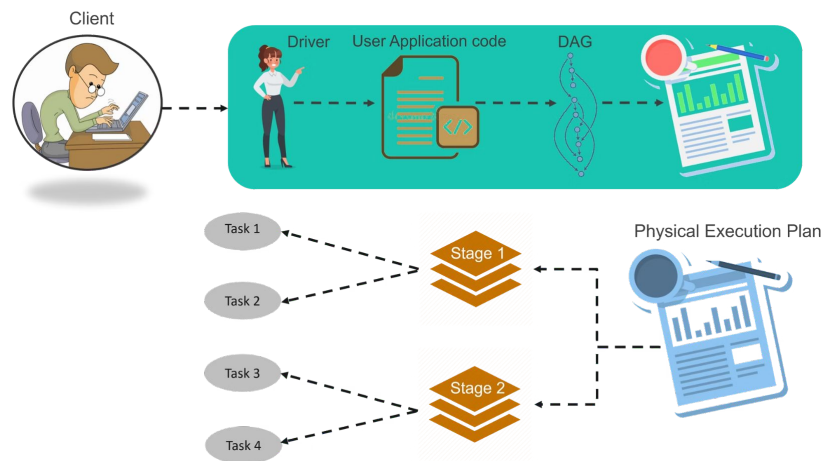
Cluster Manager

El Driver se comunica con un componente llamado **Cluster Manager**, encargado de gestionar los recursos del clúster. Existen diferentes opciones de Cluster Manager que pueden ser utilizadas con Spark, como **Standalone, Apache Mesos, Hadoop YARN** o gestores de clúster en la nube como Kubernetes.

Executors y RDDs en la arquitectura de Spark

Los nodos trabajadores del clúster son conocidos como **Executors**. Cada Executor es un proceso que ejecuta las tareas asignadas por el Driver y almacena en memoria los datos intermedios. Los Executors también son responsables de reportar el progreso de las tareas al Driver.

Una de las características clave de la arquitectura de Spark es el uso de **RDDs (Resilient Distributed Datasets)**, que permiten dividir los datos en particiones distribuidas entre los diferentes Executors. Este enfoque facilita el procesamiento paralelo y garantiza la recuperación de los datos en caso de fallos.



Executors y RDDs en la arquitectura de Spark

Executors

- Son los **workers** que ejecutan las tareas
- Procesan los datos
- Guardan datos en memoria

RDDs

- Dividen los datos en **partes (particiones)**
- Cada parte se procesa en paralelo

Los datos se distribuyen y se procesan al mismo tiempo

Jobs y Stages en la arquitectura de Spark

Proceso interno de ejecución de Spark.



1. Código de usuario

El usuario escribe código que define transformaciones y acciones sobre los datos.



2. Job

Spark organiza el procesamiento en **Jobs** y **Stages**. Un **Job** es una acción solicitada por el usuario, como contar los registros de un RDD.



3. Stage

Cada Job está compuesto por uno o más **Stages**. Cada Stage consiste en un conjunto de tareas que pueden ejecutarse en paralelo sobre las particiones de los datos.



4. Task

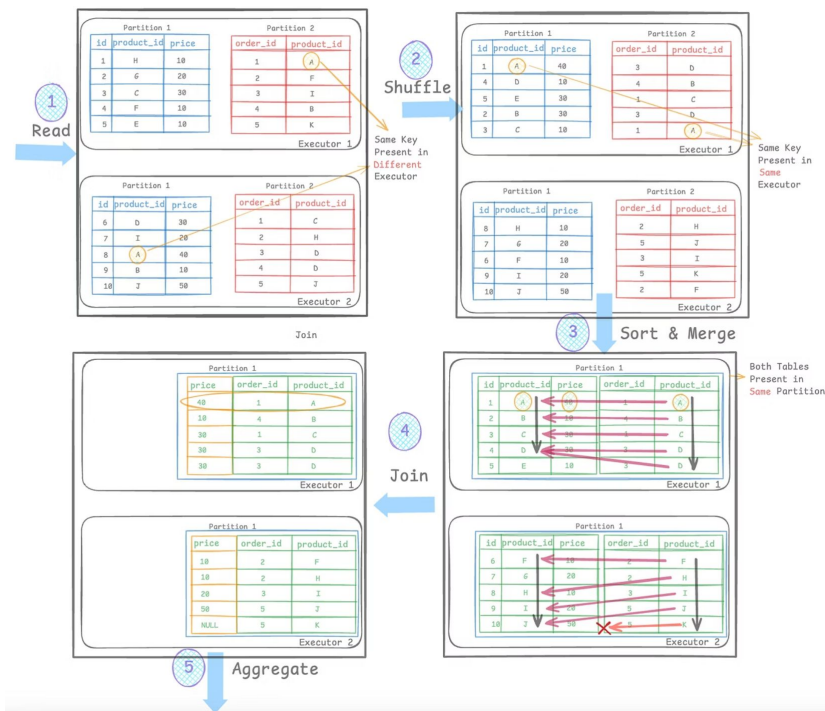
Las tareas son enviadas a los Executors para su ejecución en paralelo.

Flujo de ejecución y tolerancia a fallos

El flujo general de ejecución en Spark comienza

1. Cuando el usuario envía una acción.
2. El Driver genera un Job y lo divide en Stages.
3. Cada Stage se divide en tareas que son enviadas a los Executors para su ejecución.
4. Los resultados son recolectados y enviados de vuelta al Driver.

La arquitectura de Spark también incluye mecanismos de **tolerancia a fallos**. Si un Executor falla, las tareas asignadas pueden ser reasignadas a otro Executor utilizando la información del lineage de los RDDs.



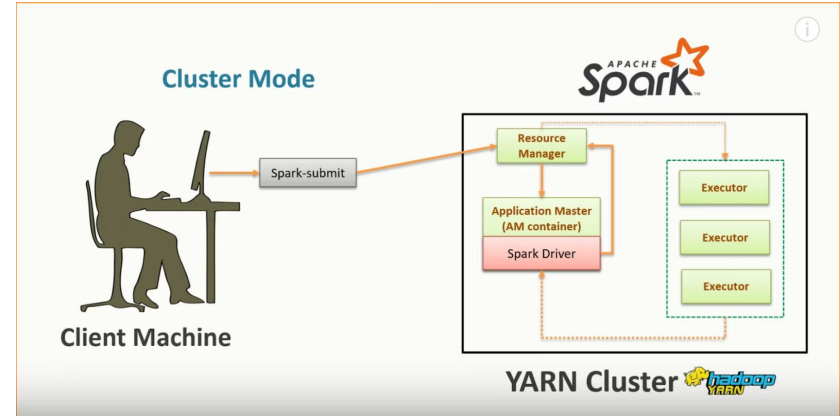
Principales componentes de Spark: Driver

La arquitectura de **Apache Spark** está formada por varios componentes esenciales que trabajan juntos para garantizar el procesamiento eficiente y distribuido de los datos.

Los tres elementos clave de esta arquitectura son el **Driver**, los **Executors** y el **Cluster Manager**.

El **Driver** es el componente principal que inicia y gestiona la ejecución de una aplicación Spark. Se encarga de crear el **SparkContext**, que es el punto de entrada para cualquier aplicación Spark, y de coordinar la ejecución de las tareas en el clúster.

El Driver también realiza el seguimiento del estado de la aplicación y recopila los resultados obtenidos por los Executors.



Principales componentes de Spark: DAG y Stages

Además, el Driver transforma las operaciones solicitadas por el usuario en un **Directed Acyclic Graph (DAG)**, que representa las dependencias entre las tareas y permite optimizar el plan de ejecución. Este DAG es dividido en etapas (Stages) que serán distribuidas y ejecutadas por los nodos trabajadores.

Principales componentes de Spark:

Executors

Definición de Executors

Los **Executors** son procesos que se ejecutan en los nodos del clúster y son responsables de realizar las tareas asignadas por el Driver. Cada Executor procesa una partición de los datos, almacena en memoria los datos intermedios y devuelve los resultados al Driver una vez completadas las tareas. Los Executors también informan al Driver sobre el estado y el progreso de las tareas.

Configuración de Executors

La eficiencia de Spark depende en gran medida de la correcta configuración y utilización de los Executors. Es posible ajustar el número de Executors, la cantidad de memoria asignada y la cantidad de núcleos utilizados para optimizar el rendimiento de las aplicaciones.

Principales componentes de Spark:

Cluster Manager

El tercer componente clave es el **Cluster Manager**, que gestiona los recursos físicos del clúster y asigna los recursos necesarios a las aplicaciones Spark.

El Cluster Manager se encarga de iniciar y monitorear los Executors, asegurando que las tareas se distribuyan equitativamente entre los nodos.



Standalone

Es el gestor de recursos nativo de Spark, simple y fácil de configurar.



Apache Mesos

Un gestor de clústeres general que permite compartir recursos entre diferentes aplicaciones distribuidas.



Hadoop YARN

Un gestor de recursos diseñado para entornos Hadoop, que permite ejecutar aplicaciones Spark junto con otras aplicaciones Hadoop.



Kubernetes

Un sistema de orquestación de contenedores que permite desplegar y gestionar aplicaciones Spark en entornos basados en contenedores.

Principales componentes de Spark:

Cluster Manager

Un Cluster Manager es como un sistema que administra computadores
Decide quién usa CPU, memoria y en qué máquina se ejecuta cada cosa

Imagina que tienes:

3 computadores
quieres usarlos juntos para procesar datos

Pregunta: ¿quién decide en qué computador corre cada tarea?

necesitas un “coordinador”

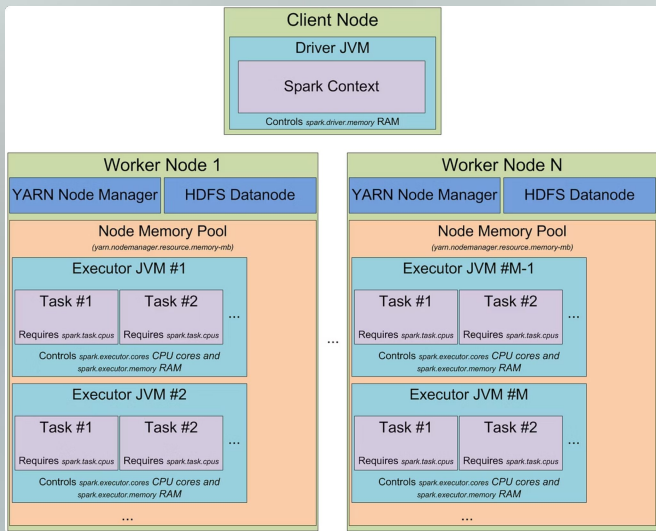
- Standalone es el coordinador más simple que trae Spark por defecto
- Apache Mesos es un sistema que administra recursos para múltiples aplicaciones, no solo Spark
- YARN es el gestor de recursos de Hadoop
- Kubernetes es un sistema que administra contenedores en un clúster

Interacción entre componentes de Spark

La interacción entre el Driver, los Executors y el Cluster Manager permite a Spark distribuir el procesamiento de los datos, garantizar la tolerancia a fallos y escalar horizontalmente para gestionar volúmenes de datos masivos.

En conclusión, comprender el rol y funcionamiento de estos tres componentes es fundamental para diseñar, configurar y ejecutar aplicaciones Spark de manera eficiente y efectiva.

Modo de funcionamiento en clúster

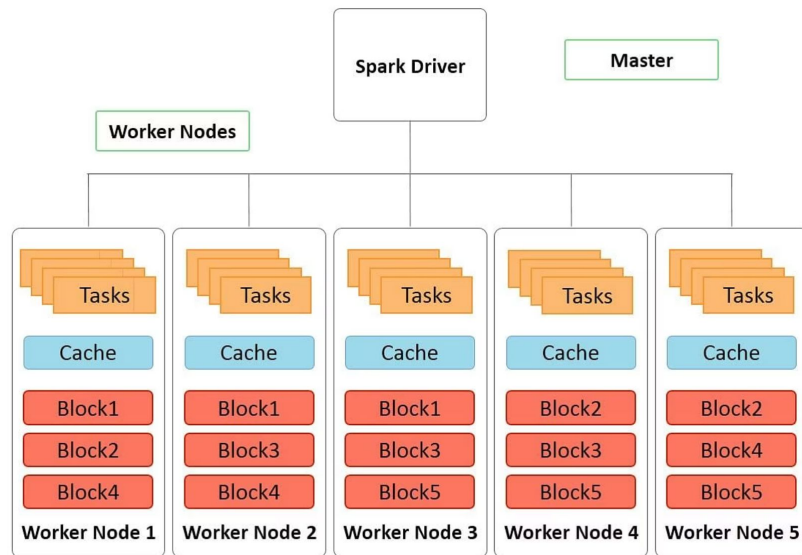


El verdadero potencial de **Apache Spark** se despliega cuando se ejecuta en un entorno distribuido, conocido como **modo clúster**.

Este modo permite procesar grandes volúmenes de datos utilizando múltiples nodos, aprovechando la potencia de cómputo y la capacidad de almacenamiento de todos los servidores que forman parte del clúster.

Distribución de aplicaciones en modo clúster

Cuando Spark se ejecuta en modo clúster, la aplicación se distribuye entre un nodo maestro y varios nodos trabajadores. El nodo maestro aloja el **Driver Program**, que coordina la ejecución de la aplicación, mientras que los nodos trabajadores ejecutan los **Executors**, encargados de procesar los datos y realizar las tareas asignadas.



Opciones de despliegue en modo clúster

1

Spark Standalone

El modo más sencillo, que utiliza un gestor de clúster propio de Spark.

- Fácil configuración y despliegue
- Ideal para entornos dedicados a Spark
- Menor flexibilidad para compartir recursos

2

Hadoop YARN

Integra Spark con un ecosistema Hadoop existente, permitiendo compartir recursos con otras aplicaciones.

- Buena integración con HDFS
- Gestión eficiente de recursos
- Soporte para múltiples aplicaciones

3

Apache Mesos

Un gestor de clúster más general, que soporta múltiples tipos de aplicaciones distribuidas.

- Alta flexibilidad
- Soporte para contenedores
- Escalabilidad avanzada

4

Kubernetes

Permite desplegar Spark en entornos de contenedores, facilitando la orquestación y escalabilidad.

- Gestión basada en contenedores
- Despliegue automatizado
- Integración con CI/CD

Flujo de trabajo en modo clúster

Envío de la aplicación

El flujo de trabajo en modo clúster comienza cuando un usuario envía una aplicación Spark al Cluster Manager.

División de tareas

El Driver divide las operaciones solicitadas en tareas y las distribuye a los Executors para su ejecución.

Asignación de recursos

El Cluster Manager asigna los recursos necesarios y lanza los Executors en los nodos trabajadores.

Ejecución y recolección

Los Executors procesan las tareas y devuelven los resultados al Driver.

Ventajas del modo clúster

Escalabilidad horizontal

Una de las ventajas del modo clúster es la **escalabilidad horizontal**. Es posible añadir más nodos al clúster para incrementar la capacidad de procesamiento y almacenamiento de datos, adaptándose al crecimiento de la demanda.

Tolerancia a fallos

Además, el modo clúster proporciona **tolerancia a fallos**. Si un nodo falla durante la ejecución, el Cluster Manager puede reasignar las tareas a otros nodos disponibles, garantizando la continuidad del procesamiento.

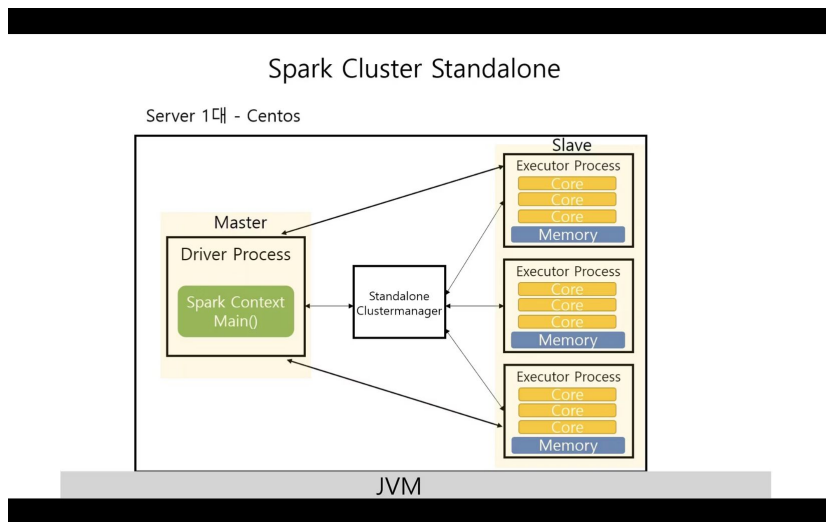
Optimización de recursos

El modo clúster también permite configurar diferentes políticas de asignación de recursos y balanceo de carga, optimizando el uso de la infraestructura disponible y asegurando que ninguna aplicación consuma más recursos de los necesarios.

Consideraciones para el modo clúster

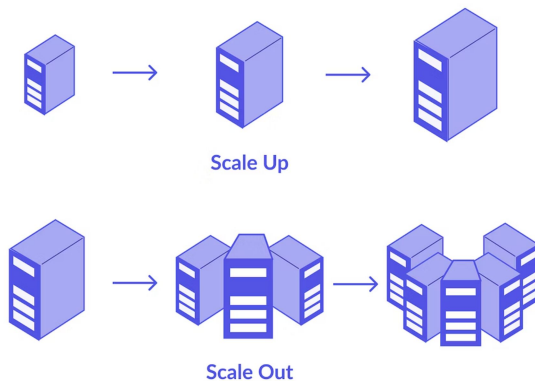
Sin embargo, la ejecución de Spark en modo clúster requiere una correcta planificación y configuración.

Aspectos como la asignación de memoria y núcleos, la ubicación de los datos y la seguridad deben ser considerados para garantizar un rendimiento óptimo y la protección de la información.



Conclusión sobre el modo clúster

En conclusión, el modo de funcionamiento en clúster es fundamental para aprovechar la verdadera potencia de Spark, permitiendo procesar grandes volúmenes de datos de manera distribuida, eficiente y escalable.



Implementaciones de Spark: Spark en la nube

El ecosistema de **Apache Spark** ha evolucionado para adaptarse a diferentes entornos y necesidades, dando lugar a diversas implementaciones que facilitan su adopción y uso en distintos contextos. Entre las más destacadas se encuentran **Spark en la nube** y **PySpark**.

Una de las implementaciones más utilizadas es **Spark en la nube**. Las principales plataformas de computación en la nube, como **Amazon Web Services (AWS)**, **Google Cloud Platform (GCP)** y **Microsoft Azure**, ofrecen servicios gestionados de Apache Spark, eliminando la necesidad de que las organizaciones configuren y administren la infraestructura subyacente.

			
 Compute Services	 Elastic Compute Cloud (EC2)	 Virtual Machines	 Compute Engine
 Object Storage	 Amazon S3	 Azure Blob Storage	 Cloud Storage
 Networking	 Amazon VPC	 Azure Virtual Network	 Cloud Virtual Network

Servicios de Spark en la nube



aws

Amazon EMR

En **AWS**, el servicio **Amazon EMR (Elastic MapReduce)** permite desplegar clústeres Spark de forma rápida y escalable, integrándolos fácilmente con otros servicios de almacenamiento y análisis de datos de la nube.



G

Google Dataproc

En **Google Cloud**, **Dataproc** proporciona un servicio similar, facilitando la ejecución de Spark con integración nativa en el ecosistema Google.



Azure

Azure HDInsight

Por su parte, **Azure HDInsight** ofrece un entorno optimizado para Spark con integración en los servicios de Microsoft.

Ventajas de Spark en la nube



Escalabilidad dinámica

Permite a las organizaciones **escalar dinámicamente** los recursos según la demanda, ajustando la capacidad de procesamiento en tiempo real.



Integración con servicios

Facilita la integración con servicios de almacenamiento, bases de datos y análisis avanzados disponibles en la nube.



Modelo de pago por uso

Las organizaciones pueden pagar únicamente por lo que consumen, reduciendo los costos de infraestructura.



Reducción de complejidad

Reduce la complejidad operativa al eliminar la necesidad de gestionar la infraestructura subyacente.

Implementaciones de Spark: PySpark

Otra implementación clave es **PySpark**, la interfaz de Apache Spark para el lenguaje de programación **Python**. PySpark permite a los desarrolladores y analistas de datos escribir aplicaciones Spark utilizando Python, uno de los lenguajes más populares en el ámbito de la ciencia de datos.

Características de PySpark

Acceso completo a Spark

PySpark ofrece acceso a todas las funcionalidades de Spark, incluyendo la creación y manipulación de **RDDs**, **DataFrames** y **Datasets**, así como el uso de módulos avanzados como **Spark SQL**, **Spark Streaming**, **MLlib** y **GraphX**. Esto facilita la integración de Spark en flujos de trabajo de análisis de datos y aprendizaje automático basados en Python.

Integración con ecosistema Python

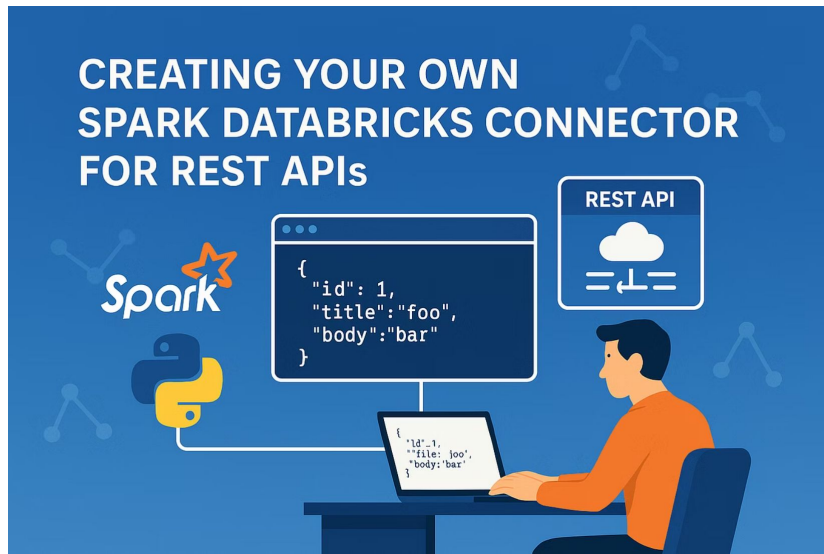
Uno de los beneficios de PySpark es que permite combinar la potencia del procesamiento distribuido de Spark con la riqueza del ecosistema Python, incluyendo librerías como **Pandas**, **NumPy** y **Scikit-learn**. Esto resulta especialmente útil para los equipos de ciencia de datos que ya trabajan con estas herramientas.

Desarrollo y pruebas simplificados

Además, PySpark simplifica el desarrollo y la prueba de aplicaciones Spark, ya que permite ejecutar código localmente antes de escalarlo a un clúster distribuido.

Consideraciones sobre PySpark

Sin embargo, es importante tener en cuenta que, al ejecutarse sobre la **Java Virtual Machine (JVM)**, PySpark puede presentar cierta sobrecarga y menor rendimiento en comparación con aplicaciones escritas directamente en Scala o Java. A pesar de ello, su facilidad de uso y compatibilidad con el ecosistema Python lo convierten en una opción muy popular.



Ejemplo de código PySpark

```
from pyspark.sql import SparkSession

# Iniciar una sesión de Spark
spark = SparkSession.builder \
    .appName("Ejemplo PySpark") \
    .getOrCreate()

# Crear un DataFrame a partir de un archivo CSV
df = spark.read.csv("datos.csv", header=True, inferSchema=True)

# Realizar transformaciones
resultado = df.filter(df.edad > 25) \
    .groupBy("departamento") \
    .count()

# Mostrar resultados
resultado.show()

# Cerrar la sesión
spark.stop()
```

Otras implementaciones de Spark

Spark en Kubernetes

Permite desplegar aplicaciones Spark en entornos de contenedores orquestados por Kubernetes, facilitando la portabilidad y la gestión de recursos.

Spark en Edge Computing

Implementaciones adaptadas para ejecutar Spark en dispositivos de borde, permitiendo el procesamiento de datos cerca de donde se generan.

Spark con GPU

Versiones optimizadas para aprovechar la potencia de procesamiento de las unidades de procesamiento gráfico, especialmente útiles para tareas de aprendizaje profundo.

Conclusión

A lo largo de este manual, hemos explorado los aspectos fundamentales de **Apache Spark**, una de las herramientas más potentes y utilizadas en el ecosistema Big Data. Comprendimos su origen y la necesidad que impulsó su desarrollo como respuesta a las limitaciones de tecnologías anteriores como Hadoop MapReduce.

Abordamos los conceptos básicos de Spark, incluyendo su modelo de programación, estructuras de datos y componentes esenciales. También analizamos los problemas que Spark resuelve y lo comparamos con otros sistemas de procesamiento, identificando los escenarios ideales para su implementación.

Además, estudiamos en detalle la arquitectura general de Spark, sus principales componentes y su modo de funcionamiento en clúster, aspectos clave para entender su escalabilidad y eficiencia. Finalmente, revisamos sus implementaciones más relevantes, como **Spark en la nube** y **PySpark**, que han facilitado su adopción en múltiples contextos.

El conocimiento adquirido en este manual proporciona una base sólida para que los estudiantes y profesionales puedan desarrollar, desplegar y gestionar soluciones basadas en Spark, aprovechando su capacidad para procesar datos masivos de manera rápida, flexible y escalable.

Live Coding

¿En qué consistirá la Demo?

Esta vez no haremos una codificación real, sino que abordaremos el funcionamiento de Spark desde una perspectiva técnica aplicada, investigando y esquematizando su ejecución.

1. ¿Cómo se transforma una operación en Spark en un DAG y luego en Stages y Tasks?
2. ¿Qué sucede en la arquitectura cuando una aplicación Spark se ejecuta en modo clúster?
3. ¿Cómo se comunican el Driver y los Executors? ¿Qué pasa ante un fallo?
4. ¿Cuáles son las diferencias entre usar Spark en Standalone vs YARN vs Kubernetes?
5. ¿Qué beneficios concretos ofrece PySpark frente a otros lenguajes?

< > < > < > < > < > < > < > < > < > < > 1. ¿Cómo se transforma una operación en Spark en un DAG y luego en Stages y Tasks?

De una operación a ejecución distribuida

1. **Operación del usuario:** `df.filter(df.edad > 25).groupBy("departamento").count().show()`
2. **DAG (Directed Acyclic Graph):** Spark construye un plan lógico con las transformaciones.
3. **Stages:** El DAG se divide en etapas según dependencias y operaciones de shuffle.
4. **Tasks:** Cada stage se divide en tareas pequeñas, una por partición.

Idea clave: Acción → Job → Stages → Tasks

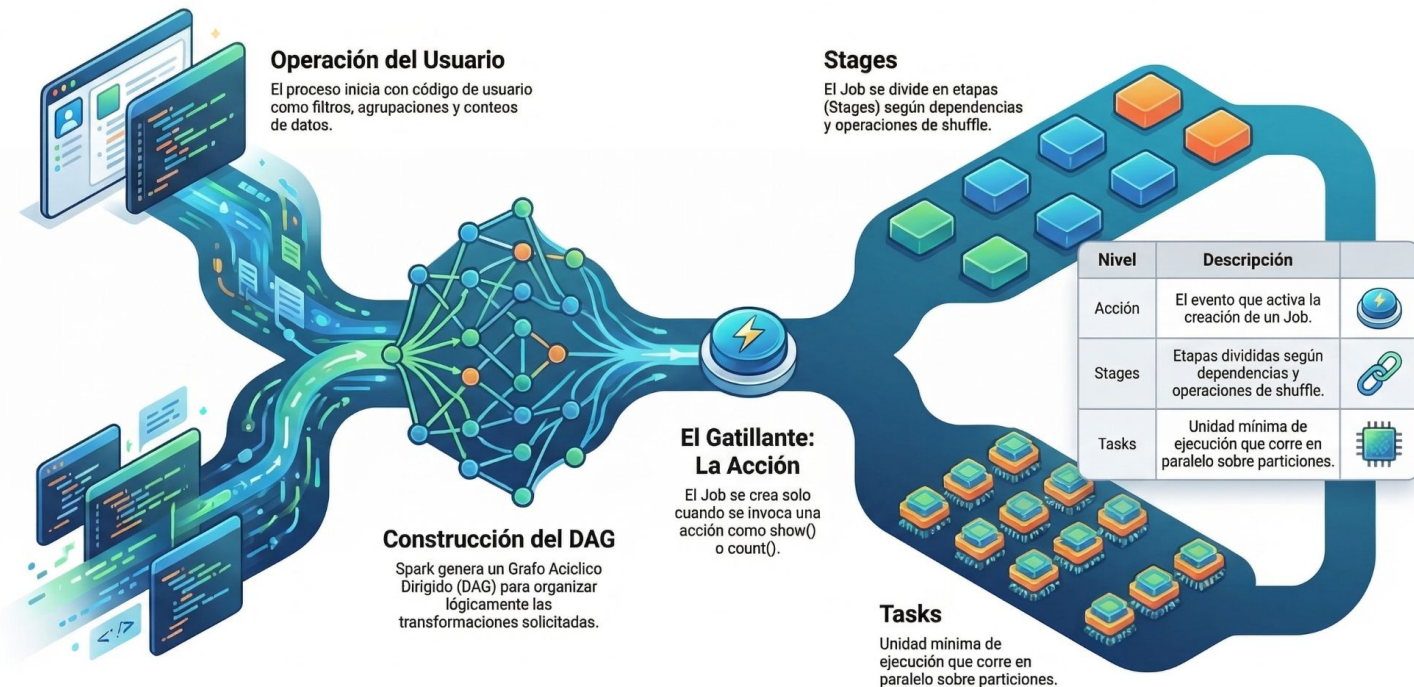
Cuando escribimos transformaciones, Spark arma un plan llamado DAG.

Ese plan no se ejecuta de inmediato.

Cuando aparece una acción, como `show()` o `count()`, Spark crea un Job.

Luego divide ese Job en Stages, y cada Stage en Tasks que se ejecutan en paralelo sobre las particiones de los datos.

El Ciclo de Ejecución en Apache Spark



2 . ¿Qué sucede en la arquitectura cuando una aplicación Spark se ejecuta en modo clúster?

Componentes:

- **Driver:** coordina la aplicación
- **Cluster Manager:** asigna recursos
- **Executors:** ejecutan tareas en los nodos

Flujo:

1. El usuario envía la aplicación
2. Se inicia el Driver
3. El Cluster Manager asigna recursos
4. Se levantan Executors
5. El Driver envía Tasks a los Executors
6. Los Executors procesan y devuelven resultados

En modo clúster, Spark no corre todo en una sola máquina.

El Driver coordina la aplicación, el Cluster Manager decide dónde asignar CPU y memoria, y los Executors se levantan en distintos nodos para ejecutar las tareas.

Eso permite que el procesamiento sea distribuido y paralelo.

3. ¿Cómo se comunican el Driver y los Executors? ¿Qué pasa ante un fallo?

Comunicación:

- El Driver envía Tasks
- Los Executors reportan estado y resultados
- El Driver coordina reintentos si algo falla

Si falla un Executor:

- Las tareas se reprograman
- Spark recalcula datos usando **lineage**
- No siempre es necesario reiniciar toda la aplicación

El Driver no procesa directamente los datos; su rol es coordinar.

Envía tareas a los Executors, recibe reportes de avance y recopila resultados parciales o finales.

Si un Executor falla, Spark puede volver a ejecutar esas tareas en otro Executor.

Esto es posible gracias al lineage, que registra cómo se construyó cada RDD o DataFrame.

4. ¿Cuáles son las diferencias entre usar Spark en Standalone vs YARN vs Kubernetes?

Opción	Qué es	Cuándo se usa	Idea clave
Standalone	Gestor propio de Spark	Laboratorios, pruebas, entornos simples	Spark se administra a sí mismo
YARN	Gestor de recursos de Hadoop	Empresas con ecosistema Hadoop	Spark usa la infraestructura de Hadoop
Kubernetes	Orquestador de contenedores	Cloud y entornos modernos	Spark corre sobre contenedores

5. ¿Qué beneficios concretos ofrece PySpark frente a otros lenguajes?

Lenguaje	Ventaja	Desventaja
Python / PySpark	Más fácil de aprender y adoptar	Puede tener overhead frente a Scala en algunos casos
Scala	Integración nativa con Spark, mejor rendimiento en ciertos escenarios	Curva de aprendizaje mayor
Java	Fuerte en entornos empresariales	Código más verboso
R	Cómodo para perfiles estadísticos	Menos usado en ingeniería de datos



Ejercicio

Mapea el flujo interno de Spark

Mapea el flujo interno de Spark

Contexto: 🙌

Te convertirás en arquitecto/a de datos y deberás comprender cómo Spark distribuye el trabajo internamente y en qué escenarios conviene usar ciertas implementaciones.

Consigna: 📝

1. Dibuja o esquematiza el flujo de ejecución de Spark desde que se lanza una acción hasta que se completan las tareas.
2. Describe el rol de cada componente: Driver, Executors, Cluster Manager.
3. Elige una implementación de Spark (AWS EMR, PySpark, Kubernetes, etc.) y explica por qué y cuándo conviene usarla.
4. Enumera tres buenas prácticas para correr Spark en modo clúster de forma eficiente.

Paso a paso: ⚙️

1. Crea un gráfico simple (puede ser en papel o digital).
2. Escribe una breve justificación para tu implementación elegida.
3. Prepara un mini-pitch de 2 minutos para presentarlo en clase.

Tiempo 🕒: 45 Minutos

¿Alguna consulta?





¡Ponte a prueba!

Momento de ejercitación

*Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.*

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< ¡Muchas gracias! >

